

Documentação Técnica

Acompanhamento de Atividade e Juros — Brasil e EUA

Como funciona o robô automatizado que baixa, processa e publica mensalmente indicadores de produção industrial e juros de curto prazo para Brasil e EUA.

1. Visão geral

O projeto automatiza a produção de um relatório mensal com indicadores macroeconômicos de Brasil e Estados Unidos. Toda a estrutura roda na nuvem do GitHub e não exige nenhum computador ligado: no dia 5 de cada mês, às 9h (horário de Brasília), os dados são baixados, processados, e os arquivos atualizados são publicados de volta no repositório.

Indicadores acompanhados:

- Produção industrial (com e sem ajuste sazonal) — Brasil e EUA
- Fed Funds Rate — EUA
- Taxa Selic (meta) — Brasil

Fontes de dados:

País	Indicador	Fonte	Código
EUA	Produção industrial (SA)	FRED	INDPRO
EUA	Produção industrial (NSA)	FRED	IPB50001N
EUA	Fed Funds Rate	FRED	FEDFUNDS
BR	Produção industrial (SA)	BCB SGS	21859
BR	Produção industrial (NSA)	BCB SGS	28503
BR	Selic (meta)	BCB SGS	432

Por que duas fontes?

O FRED (Federal Reserve Bank of St. Louis) é a referência para séries dos EUA e tem uma API muito estável. Para o Brasil, usamos o sistema SGS do Banco Central, que disponibiliza séries macro brasileiras em JSON. O FRED exige chave de API (gratuita); o BCB não exige.

2. Arquitetura do projeto

O projeto vive em um repositório no GitHub e tem três componentes principais que conversam entre si:

Componentes:

script.py — código Python que baixa, processa e exporta os dados.

.github/workflows/relatorio.yml — instruções do GitHub Actions sobre quando e como rodar o script.

Repository Secrets — onde fica guardada a chave de API do FRED, em formato criptografado.

Fluxo de execução:

1. O GitHub Actions é acionado (pelo agendamento mensal ou por botão manual).
2. Uma máquina virtual Linux é criada do zero, com Python 3.12 instalado.
3. As dependências são instaladas (pandas, requests, matplotlib, openpyxl, python-dateutil).
4. O script.py é executado, lendo a chave do FRED a partir do Secret.
5. Os arquivos gerados (.xlsx, .csv, .png) são commitados de volta no repositório.
6. Os mesmos arquivos ficam disponíveis para download como artefatos do run.

3. O script Python (script.py)

O script é dividido em blocos lógicos. Cada bloco tem uma responsabilidade clara.

3.1 Configuração inicial

No topo do arquivo definimos a chave de API, os cabeçalhos HTTP, os códigos das séries e o intervalo de datas. A chave nunca é escrita no código — ela é lida de uma variável de ambiente injetada pelo GitHub Actions.

```
FRED_API_KEY = os.environ["FRED_API_KEY"]

HEADERS = {
    "User-Agent": "Mozilla/5.0 (relatorio-eesp-quant)",
    "Accept": "application/json",
}

FRED_SERIES = {
    "ind_prod_sa": "INDPRO",
    "ind_prod_nsa": "IPB50001N",
    "fed_funds": "FEDFUNDS",
}

BCB_SERIES = {
    "ind_prod_sa": 21859,
    "ind_prod_nsa": 28503,
    "selic": 432,
}
```

O User-Agent é obrigatório para o BCB — a API deles retorna respostas vazias se a requisição não tiver esse cabeçalho.

3.2 Função de download resiliente — `_get()`

Toda chamada HTTP passa por essa função. Ela existe porque APIs públicas falham com frequência: podem dar timeout, retornar vazio, retornar HTML em vez de JSON, ou ficar fora do ar por alguns segundos. A função tenta até 3 vezes com espera crescente entre tentativas. Se mesmo assim falhar, retorna None em vez de quebrar o script.

```
def _get(url, retries=3, timeout=60):
    for tentativa in range(1, retries + 1):
        try:
            r = requests.get(url, headers=HEADERS, timeout=timeout)
            if r.status_code == 404 or not r.text.strip():
                return None
            if r.status_code >= 400:
                time.sleep(3 * tentativa); continue
            return r.json()
        except (requests.exceptions.ReadTimeout,
                requests.exceptions.ConnectionError,
                ValueError):
            time.sleep(3 * tentativa)
    return None
```

3.3 Coleta do FRED — `get_fred()`

Monta a URL da API do FRED com a chave, série e período. O FRED retorna um JSON com uma lista de observações; a função converte para uma série do pandas indexada por data.

3.4 Coleta do BCB — `get_bcb()`

A API do BCB tem um detalhe chato: quando o intervalo de datas é muito longo, ela trava ou devolve resposta vazia. A solução foi quebrar o pedido em pedaços de 10 anos. A função percorre a janela de tempo em blocos, baixa cada pedaço e concatena tudo no final, removendo duplicatas nas bordas.

3.5 Análise e estatísticas descritivas

Depois de baixar e organizar tudo em DataFrames mensais (com `.resample("ME").last()`), o script calcula estatísticas em três níveis:

Por série individual: média, desvio padrão, mínimo, máximo, quartis (função `describe`).

Comparação SA vs NSA: correlação entre a série ajustada e não-ajustada, diferença média, desvio da diferença, e variação interanual média de cada uma.

Cobertura temporal: primeiro e último período disponível, número total de observações.

3.6 Geração de gráficos

O matplotlib monta um painel 2x2 com produção industrial e juros para cada país. A função `plot_safe` verifica se há dados antes de plotar — assim, se uma série vier vazia por algum motivo, o painel mostra '(sem dados)' no título em vez de quebrar.

3.7 Exportação

Os dados saem em quatro formatos para diferentes usos:

relatorio_atividade_juros.xlsx — Excel com 4 abas (EUA, Brasil, Estatísticas, Comparação SA-NSA).

eua.csv / brasil.csv — séries históricas em texto, fáceis de abrir em qualquer ferramenta.

estatisticas.csv / comparacao_sa_nsa.csv — tabelas resumo para citação rápida.

relatorio.png — imagem do painel de gráficos.

4. O workflow do GitHub Actions

Esse arquivo YAML diz ao GitHub quando e como rodar o script. Cada bloco tem uma função.

4.1 Quando rodar — bloco 'on:'

```
on:
  schedule:
    - cron: "0 12 5 * *"
  workflow_dispatch:
```

A expressão cron `0 12 5 * *` significa: minuto 0, hora 12 (UTC), dia 5, qualquer mês, qualquer dia da semana. Em horário de Brasília (UTC-3), isso equivale a 9h da manhã do dia 5. O `workflow_dispatch` habilita o botão para rodar manualmente.

4.2 O que rodar — bloco 'jobs:'

O job é uma sequência de passos executados numa máquina virtual Ubuntu:

checkout — clona o repositório dentro da máquina.

setup-python — instala Python 3.12.

Instalar dependências — `pip install` das bibliotecas.

Rodar script — executa `python script.py`, com a chave do FRED injetada via `env`.

Salvar arquivos no repositório — `git add`, `commit` e `push` dos arquivos gerados.

Disponibilizar para download — empacota os arquivos como artifact do run.

4.3 Permissões

A linha `permissions: contents: write` autoriza o GitHub Actions a fazer `commit` no próprio repositório. Sem isso, o passo de salvar arquivos falharia com erro de permissão.

5. Como a chave de API fica protegida

A chave do FRED não está em lugar nenhum do código. Ela é guardada como um **Repository Secret** (Settings → Secrets and variables → Actions). O GitHub criptografa o valor e só o expõe como variável de ambiente durante a execução do workflow. Mesmo nos logs, o valor aparece mascarado como ***.

Isso garante que, mesmo se o repositório for público, ninguém consegue ler a chave nem usá-la indevidamente.

6. Operação e manutenção

Como rodar manualmente:

Aba **Actions** → no menu esquerdo clicar em **Relatório Mensal** → botão **Run workflow** no canto direito → confirmar com Branch: main.

Como saber se rodou com sucesso:

Na aba Actions, cada run mostra um ícone: ■ verde indica sucesso, ■ vermelho indica falha, ■ amarelo indica em execução. Em caso de falha, o GitHub envia automaticamente um e-mail para o endereço associado à conta.

Como diagnosticar um erro:

Clicar no run vermelho → entrar no job **run** → expandir o passo marcado com ■. A mensagem em vermelho no fim do log indica a causa. Erros comuns: API fora do ar (basta rerodar), série descontinuada (precisa trocar o código), chave expirada (gerar nova no FRED e atualizar o Secret).

O agendamento pode ser desativado:

O GitHub desativa workflows agendados se o repositório fica 60 dias sem nenhuma atividade. Se isso acontecer, basta aparecer na aba Actions e clicar em **Enable workflow**. Qualquer commit no repo zera esse contador.

7. Resumo em uma frase

Todo dia 5 do mês, uma máquina virtual no GitHub baixa dados macroeconômicos do FRED e do BCB, calcula estatísticas comparando séries ajustadas e não-ajustadas sazonalmente, gera um painel de gráficos, e publica xlsx + csv + png de volta no repositório — tudo sem nenhum computador do usuário precisar estar ligado.